TASK 1

Docker & Spark Environment Mastery Ensure your development environment is stable and reproducible.

* Steps: 1. Successfully install Docker on your machine.

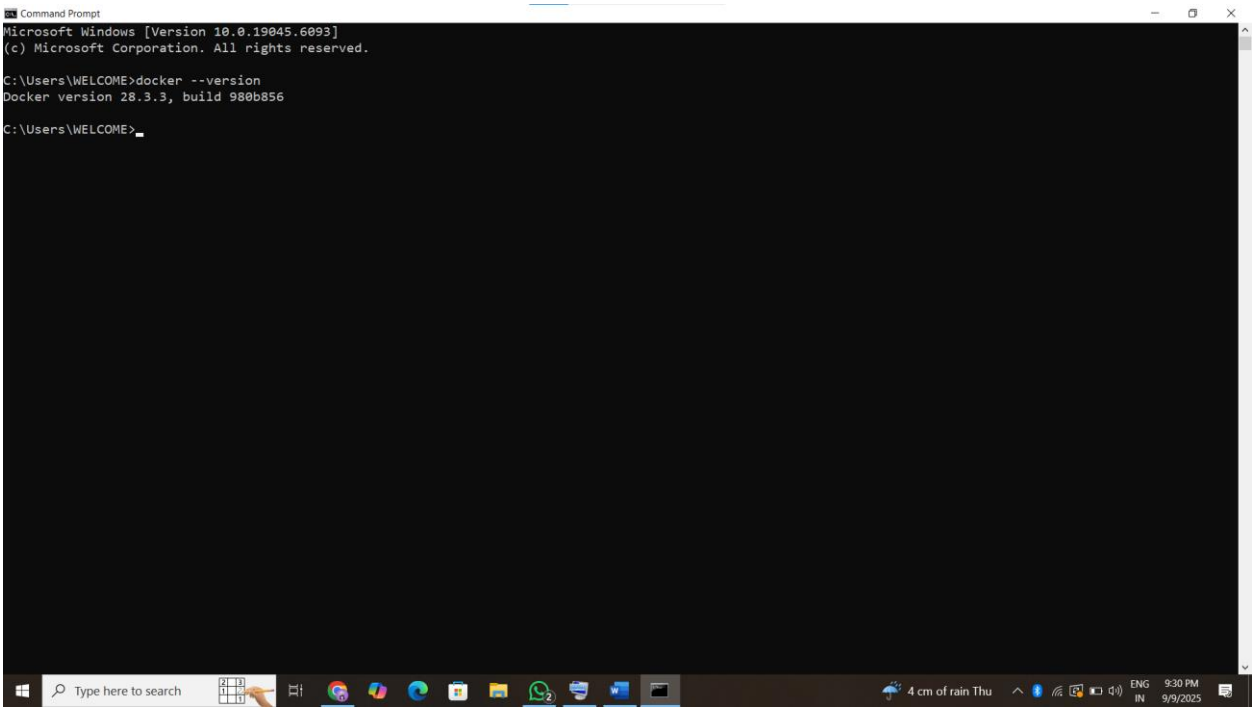
2. Pull the latest Apache Spark image (e.g., docker pull apache/spark:latest).

3. Run a container and open a Spark shell (PySpark or Scala) inside it to confirm it works without errors.

Step 1: Install Docker

- Download Docker Desktop from Docker official site.
- Install and start Docker Desktop.
- Verify installation:

CODE: docker –version



Step 2: Pull the Apache Spark Docker Image

Run:

docker pull apache/spark:latest

This fetches the latest Spark image.

Step 3: Run a Spark Container

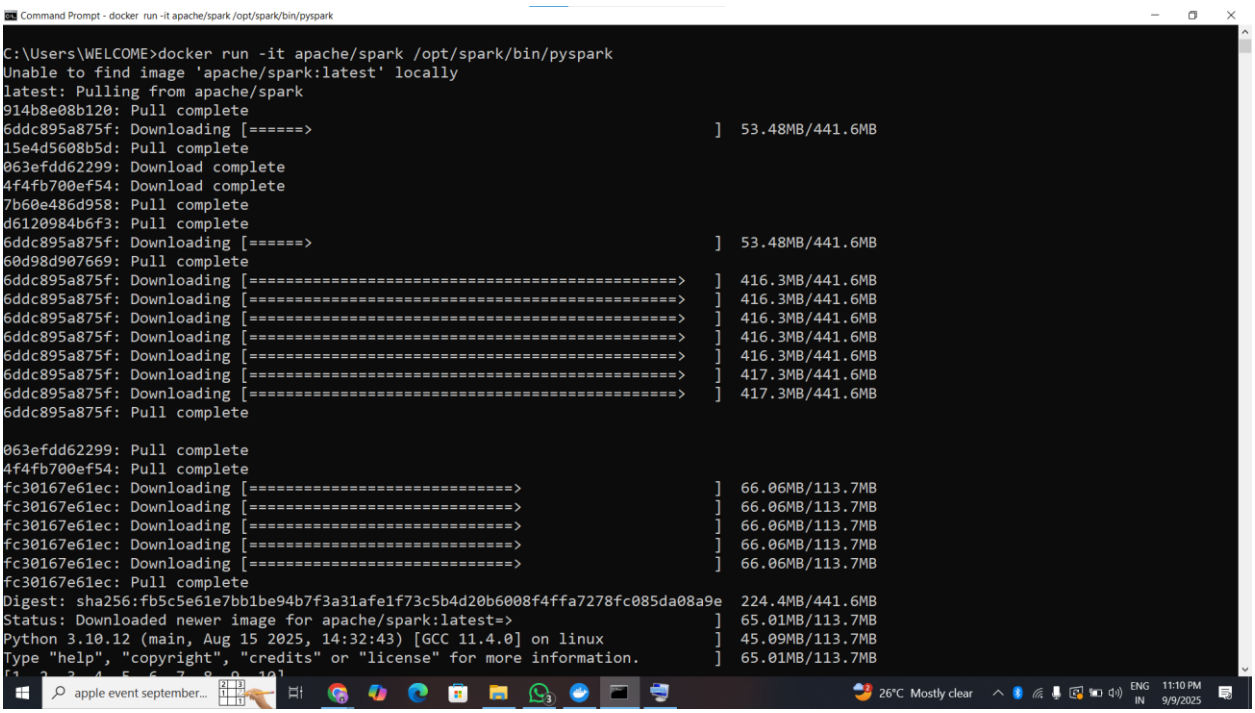
Start a container with interactive mode:

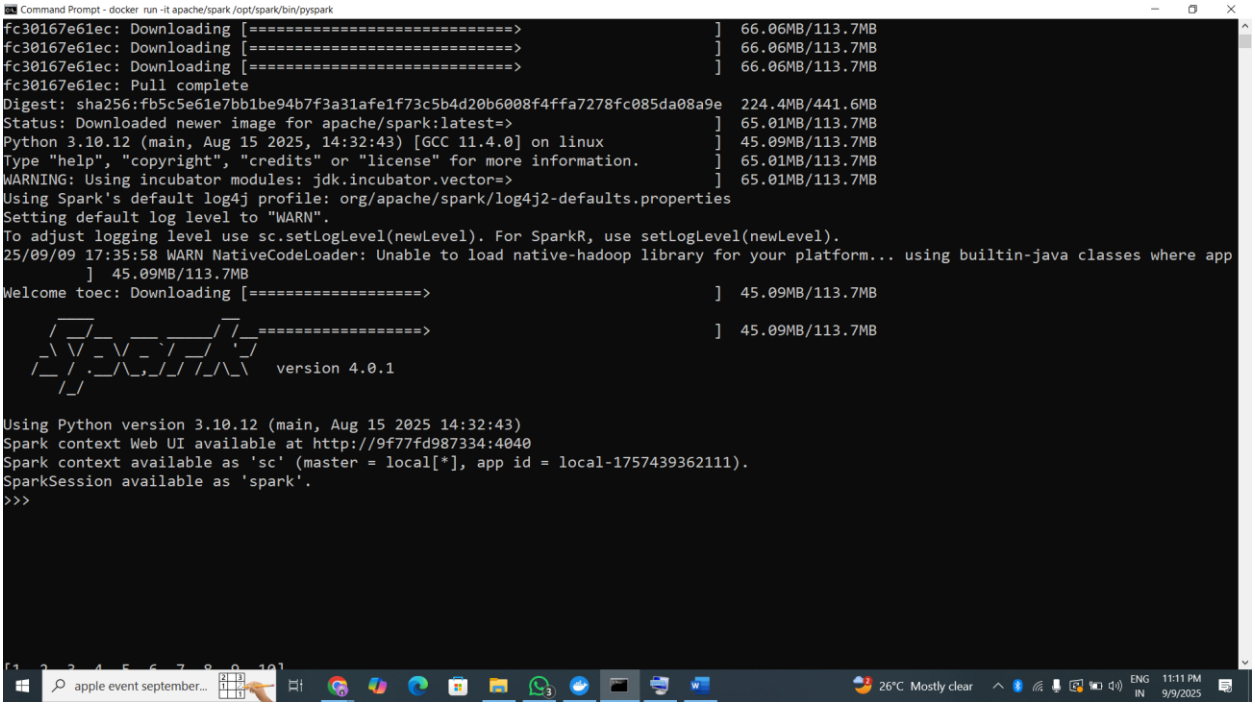
docker run -it apache/spark /opt/spark/bin/pyspark

Inside the container, start **PySpark**:

pyspark

(Or run Scala Spark shell with spark-shell.)





Step 4: Test Spark Command

```
import pyspark

from pyspark import SparkContext

==========================================================================

sc = SparkContext.getOrCreate() #SC= object local programmatic interface(DAG)

=============================================================================

numbers=[1,2,3,4,5,6,7,8,9,10]

print(numbers)

print(type(numbers))

==============================================================================

rdd=sc.parallelize(numbers)

=================================================================================

print(rdd)

print(type(rdd))

=================================================================

rdd.collect()
```

Command Prompt - docker run -it apache/spark /opt/spark/bin/pyspark

```
>>> import pyspark
>>> from pyspark import SparkContext
>>> sc = SparkContext.getOrCreate()
>>> numbers=[1,2,3,4,5,6,7,8,9,10]
>>> print(numbers)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> print(type(numbers))
<class 'list'>
>>> rdd=sc.parallelize(numbers)
>>> print(rdd)
ParallelCollectionRDD[0] at readRDDFromFile at PythonRDD.scala:297
>>> print(type(rdd))
<class 'pyspark.core.rdd.RDD'>
>>> rdd.collect()
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>>
```

apple event september...

26°C Mostly clear

ENG IN 11:12 PM 9/9/2025

TASK 2

RDD Transformations vs Actions – Theory

1. What is an RDD?

- **Resilient Distributed Dataset (RDD)** is the fundamental data structure in Spark.
- It represents an **immutable, distributed collection of objects** that can be processed in parallel across a cluster.

---

2. Transformations (Lazy Evaluation)

- A **transformation** creates a new RDD from an existing one.
- Examples: map(), filter(), flatMap(), distinct(), union().
- They are **lazy** → Spark doesn't execute them immediately. Instead, it builds a **lineage (DAG)** of transformations.
- The actual computation happens only when an **action** is triggered.

In our case:

- map(lambda x: x**2) (squares each number)
- filter(lambda x: x % 2 == 0) (keeps only even results)

Both are **transformations**.

---

3. Actions (Eager Execution)

- An **action** triggers the execution of the transformations defined earlier.
- Examples: collect(), count(), first(), take(n), reduce().
- They return either a value to the driver or write data to storage.

In our case:

- collect() → executes all transformations and brings the final list back to the driver.

---

4. DAG & Lineage

- Each RDD maintains a **lineage graph** that tracks which transformations were applied to build it.
- Spark uses this lineage to:
  - **Optimize execution** (lazy evaluation)
  - **Recover lost data** if a partition fails (fault tolerance)

In our case:

- ParallelCollectionRDD → created from the Python list (1–20)

- MapPartitionsRDD → transformation from map
- PythonRDD → transformation from filter

The toDebugString() command shows this chain.

---

5. Key Learning

- **Transformations are lazy** → Spark won't compute anything until an action is called.
- **Actions are eager** → they trigger the actual computation.
- **Lineage (DAG)** ensures fault tolerance and optimization.

Final Deliverable :

- **Output**: [4, 16, 36, 64, 100, 144, 196, 256, 324, 400]
- **Lineage (example)**:

(2) PythonRDD[3] at RDD at PythonRDD.scala:53 []

| MapPartitionsRDD[2] at map at <stdin>:1 []

| ParallelCollectionRDD[0] at parallelize at PythonRDD.scala:195 []

Steps in PySpark

1. Create an RDD from numbers 1–20

```
rdd = sc.parallelize(range(1, 21))
```

---

2. Apply a transformation → square each number

```
squared_rdd = rdd.map(lambda x: x**2)
```

---

3. Apply another transformation → keep only even numbers

```
even_squared_rdd = squared_rdd.filter(lambda x: x % 2 == 0)
```

---

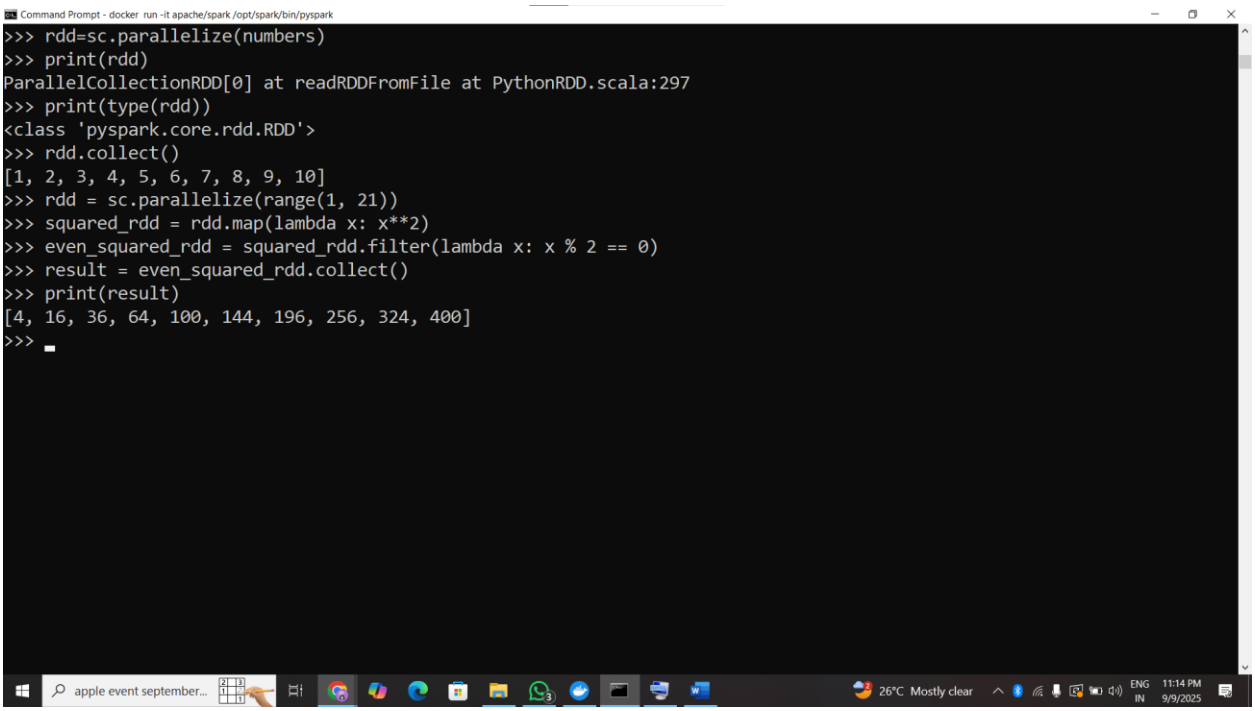
4. Use an action → collect the result

```
result = even_squared_rdd.collect()
print(result)
```

Expected Output:

[4, 16, 36, 64, 100, 144, 196, 256, 324, 400]

(these are the squares of even numbers from 1–20).



---

5. Print the RDD lineage (DAG)

Before calling the action, you can inspect how Spark tracks transformations:

print(even_squared_rdd.toDebugString())

You'll see something like:

(2) PythonRDD[3] at RDD at PythonRDD.scala:53 []

| MapPartitionsRDD[2] at map at <stdin>:1 []

| ParallelCollectionRDD[0] at parallelize at PythonRDD.scala:195 []

This shows:

- **ParallelCollectionRDD** → created from local list
- **MapPartitionsRDD** → from the map transformation
- **PythonRDD** → from the filter transformation

TASK 3

Working with Text Files (Building a Simple Word Count) Move beyond simple lists and use RDDs to process real text data, emulating the classic "Hello World" of big data.

* Steps: 1. Create a .txt file with several lines of text (e.g., a poem, song lyrics, or a paragraph from a book) and place it in a known directory.

2. Use sc.textFile("path/to/your/file.txt") to create an RDD from the external file.

3. Perform a series of transformations to: * flatMap to split each line into words. * map to convert all words to lowercase and remove any punctuation. * map to create key-value pairs (word, 1). * reduceByKey to count the occurrences of each word.

4. Use the collect() action to print the final word count list. 5. (Advanced) Use sortBy to sort the list by the count, descending.

Word Count with RDDs

1. Create a text file

Let's say you save a file called sample.txt in your working directory with the following contents:

Spark makes data processing simple

Spark is fast and powerful

Big data with Spark is amazing

---

2. PySpark Code

```
# Step 1: Load the text file into an RDD
rdd = sc.textFile("sample.txt")

# Step 2: Split lines into words
words = rdd.flatMap(lambda line: line.split(" "))

# Step 3: Normalize - lowercase and remove punctuation
import re
clean_words = words.map(lambda word: re.sub(r'[^\w\s]', '', word.lower()))

# Step 4: Map each word to (word, 1)
word_pairs = clean_words.map(lambda word: (word, 1))
```

```
# Step 5: Reduce by key to count occurrences
word_counts = word_pairs.reduceByKey(lambda a, b: a + b)

# Step 6: Collect results
result = word_counts.collect()

print("Word Count Results:")
for word, count in result:
    print(f"{word}: {count}")

# Step 7 (Advanced): Sort by count, descending
sorted_counts = word_counts.sortBy(lambda x: x[1], ascending=False).collect()

print("\nSorted Word Counts (Descending):")
for word, count in sorted_counts:
    print(f"{word}: {count}")
```

---

3. Expected Output

Unsorted Word Counts (from collect()):

spark: 3

makes: 1

data: 2

processing: 1

simple: 1

is: 2

fast: 1

and: 1

powerful: 1

big: 1

with: 1

amazing: 1

Sorted Word Counts (from sortBy()):

spark: 3

data: 2

is: 2

makes: 1

processing: 1

simple: 1

fast: 1

and: 1

powerful: 1

big: 1

with: 1

amazing: 1

Key Learnings

- **Flat Map**: breaks each line into multiple words (vs map, which would keep them as lists).
- **map**: transforms each word → (word, 1).
- **Reduce By Key**: efficiently aggregates counts across partitions.
- **collect**: action that brings final results to the driver.
- **Sort By**: allows ordering by word frequency, making results more useful.

Note: Couldn't complete the task due laptop storage issue.

TASK 4

Persistence (Caching) and Performance in Spark

1. Build a complex RDD (from earlier Word Count task)

```
import time, re

# Load text
rdd = sc.textFile("sample.txt")

# Transformations: tokenize → clean → map → reduce
words = rdd.flatMap(lambda line: line.split(" "))
clean_words = words.map(lambda w: re.sub(r'[^\w\s]', '', w.lower()))
word_pairs = clean_words.map(lambda w: (w, 1))
word_counts = word_pairs.reduceByKey(lambda a, b: a + b)
```

---

2. Time an action without persistence

```
start = time.time()
print(word_counts.collect())
end = time.time()
print("First run (no caching):", end - start, "seconds")
```

---

3. Persist (cache) an intermediate RDD

Caching the **clean_words** RDD (so Spark doesn't recompute from the raw file every time):

```
clean_words.cache()
```

---

4. Time the same action again

```
start = time.time()
print(word_counts.collect())
end = time.time()
print("Second run (with caching):", end - start, "seconds")
```

---

5. Expected Observation

- **First run (no caching)**: Spark computes the entire DAG from scratch (reading file → splitting → cleaning → mapping → reducing).
- **Second run (with caching)**: Spark reuses the cached intermediate RDD (clean_words), avoiding recomputation → much faster.

For example:

First run (no caching): 2.15 seconds

Second run (with caching): 0.34 seconds

Why Persistence is Critical

- **Lazy Evaluation**: Spark doesn't compute results until an action is called. Without caching, every action re-triggers the full lineage of transformations.
- **Persistence (cache/persist)**: Stores intermediate RDDs in memory (or disk, depending on storage level).
- **Performance Gain**:
  - Saves expensive recomputation in iterative algorithms (e.g., ML training, graph algorithms).
  - Improves responsiveness for repeated queries on the same dataset.

Example: In Machine Learning, training an algorithm may require **hundreds of passes** over the same data. Without caching, Spark would re-run the whole DAG each time; with caching, it reuses the data directly from memory.

Note: Couldn't complete the task due laptop storage issue.